

Appunti — Il Canvas HTML

Ripasso sintetico: disegno 2D con JavaScript sul tag <canvas>.

1. Cos'è il Canvas

Tag HTML che funziona come tela vuota: si disegna solo via JavaScript.

```
<canvas id="canvas" width="500" height="400"></canvas>
```

```
const canvas = document.getElementById("canvas");  
const ctx = canvas.getContext("2d");
```

Sistema di coordinate

- (0,0) è in alto a sinistra, non al centro
- X cresce verso destra
- Y cresce verso il **basso** (occhio, al contrario della matematica!)

Canvas vs SVG

- Canvas = pixel (bitmap): non ricorda le forme, va ridisegnato per cambiare qualcosa
- SVG = forme vettoriali nel DOM: ogni forma resta modificabile come un elemento HTML

Importante: width/height vanno impostati come attributi HTML, non in CSS (altrimenti il disegno si deforma).

2. Rettangoli

- **fillRect(x, y, w, h)** — rettangolo pieno
- **strokeRect(x, y, w, h)** — solo bordo
- **clearRect(x, y, w, h)** — cancella l'area (diventa trasparente)

```
ctx.fillStyle = "#4f8ef7";  
ctx.fillRect(30, 30, 120, 80);
```

```
ctx.strokeStyle = "red";  
ctx.lineWidth = 4;  
ctx.strokeRect(190, 30, 120, 80);
```

fillStyle/strokeStyle accettano: nome colore, hex (#rrggbb), rgb(), rgba() (con trasparenza).

3. Path (Percorsi)

- **beginPath()** — inizia un nuovo percorso
- **moveTo(x, y)** — sposta la penna senza disegnare
- **lineTo(x, y)** — traccia una linea
- **closePath()** — chiude tornando al punto iniziale
- **stroke()** — disegna solo il bordo
- **fill()** — riempie l'area

```
ctx.beginPath();  
ctx.moveTo(100, 20);  
ctx.lineTo(160, 120);  
ctx.lineTo(40, 120);  
ctx.closePath();
```

```
ctx.fillStyle = "orange";
ctx.fill();
```

Ricorda sempre `beginPath()` prima di una nuova forma, altrimenti si aggiunge al percorso precedente.

4. Cerchi e Archi

```
ctx.arc(x, y, raggio, angoloInizio, angoloFine, sensoAntiorario)
```

Gli angoli sono in **radianti**: 0 = destra, `Math.PI` = sinistra, `2*Math.PI` = giro completo.

Conversione gradi → radianti: **gradi * `Math.PI` / 180**

```
// cerchio pieno
ctx.beginPath();
ctx.arc(80, 90, 60, 0, 2 * Math.PI);
ctx.fill();

// spicchio: moveTo al centro prima dell'arco
ctx.beginPath();
ctx.moveTo(cx, cy);
ctx.arc(cx, cy, 60, 0, Math.PI / 2);
ctx.closePath();
ctx.fill();
```

5. Testo

- **`fillText(testo, x, y)`** — testo pieno
- **`strokeText(testo, x, y)`** — testo solo bordo
- **`font`** — es. "20px Arial"
- **`textAlign`** — left / center / right
- **`textBaseline`** — top / middle / bottom
- **`measureText(testo).width`** — larghezza del testo in pixel

```
ctx.font = "bold 28px Arial";
ctx.textAlign = "center";
ctx.textBaseline = "middle";
ctx.fillText("Ciao", canvas.width/2, canvas.height/2);
```

6. Testo Verticale

Non esiste un modo diretto: si ruota il sistema di coordinate, si scrive, si ripristina.

- **`save()`** — salva lo stato attuale del contesto
- **`translate(x, y)`** — sposta l'origine degli assi
- **`rotate(angolo)`** — ruota gli assi (radianti)
- **`restore()`** — ripristina lo stato salvato

```
ctx.save();
ctx.translate(30, 220);
ctx.rotate(-Math.PI / 2);
ctx.fillText("Quantità", 0, 0);
ctx.restore();
```

Senza `restore()` la rotazione resta attiva per sempre su tutto ciò che disegni dopo!

7. Colori e Stili

- `fillStyle/strokeStyle`: nome, hex, `rgb()`, `rgba()`
- **globalAlpha** — trasparenza globale (0 = invisibile, 1 = opaco)
- **createLinearGradient(x0,y0,x1,y1)** + `addColorStop(pos, colore)`
- **createRadialGradient(x0,y0,r0,x1,y1,r1)** + `addColorStop(pos, colore)`

```
const grad = ctx.createLinearGradient(0, 0, 300, 0);
grad.addColorStop(0, "#ff512f");
grad.addColorStop(1, "#f09819");
ctx.fillStyle = grad;
ctx.fillRect(0, 0, 300, 50);
```

8. Pulizia e Reset

```
ctx.clearRect(0, 0, canvas.width, canvas.height); // pulisce tutto
```

Pattern render(): una funzione che pulisce tutto e ridisegna da zero in base allo stato attuale, invece di modificare solo un pezzo.

```
function render() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  // ridisegno tutto in base ai dati attuali
}
```

9. Esempio Completo: Istogramma

Combina tutto quanto visto: assi, barre proporzionali, etichette con `for`, testo verticale.

- Assi con `path` (eventualmente tratteggiati con `setLineDash([5,5])`)
- Barre con `fillRect`, altezza = valore * fattore di scala
- Barra parte dal basso: $y = \text{altezzaGrafico} - \text{altezzaBarra}$ (Y cresce verso il basso!)
- Etichette voto sotto ogni barra con un `for`
- Numeri sull'asse Y con un altro `for`
- Scritta "Quantità" verticale con `save/translate/rotate/restore`

```
const max = Math.max(...voti);
const scalaY = altezzaGrafico / max;
voti.forEach((v, i) => {
  const h = v * scalaY;
  ctx.fillRect(x(i), baseY - h, largh, h);
});
```